

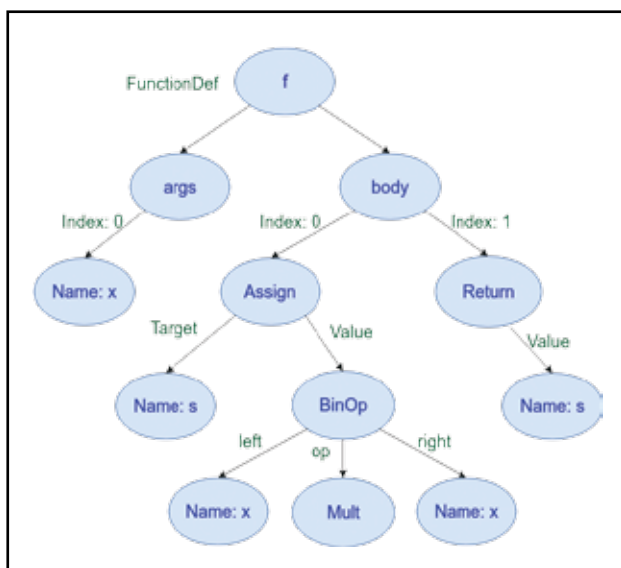
The `ast.parse` method returns an `ast_module` object which contains the AST of our function. We can print the whole tree by `ast.dump` method.

```
print ast.dump(func_ast) //prints
>>> FunctionDef(name='f', args=arguments(ar
gs=[Name(id='x', ctx=Param())], vararg=None,
kwarg=None, defaults=[]), body=[Assign(targets=[Na
me(id='s', ctx=Store())], value=BinOp(left=Name(id='x',
ctx=Load()), op=Mult(), right=Name(id='x', ctx=Load()))),
Return(value=Name(id='s', ctx=Load()))], decorator_list=[])
```

The pretty version of above output would be

```
FunctionDef(
  name='f',
  args=arguments(
    args=[
      Name(id='x', ctx=Param())
    ],
    vararg=None,
    kwarg=None,
    defaults=[]
  ),
  body=[
    Assign(
      targets=[
        Name(id='s', ctx=Store())
      ],
      value=BinOp(
        left=Name(id='x', ctx=Load()),
        op=Mult(),
        right=Name(id='x', ctx=Load())
      )
    ),
    Return(
      value=Name(id='s', ctx=Load())
    )
  ],
  decorator_list=[]
)
```

The graphical representation of above tree would be



So here is the code to compile our AST and then execute it.

```
code_object = compile(module_ast,
filename='my_code', mode='exec')
exec(code_object) # Execute the code. It cre-
ates the function f in globals context
f = eval('f') # Getting the function f from globals context
print f(4) # Calling the function. Prints 16
```

Why did we choose to use AST?

We chose to use AST manipulation to implement the profiler and the trick here is to patch every function call, attribute access and other constructs. Here is what we thought:

```
Suppose there is a function f which internally calls get_
user triggering a SQL query
@profile
def f(request):
...
    user = get_user(request)
...
```

and decorating the function `f` with `profile` decorator will manipulate its AST code. It will find all locations of function calls inside function body and will encapsulate it in our special function `call_handler`. So the function `f`'s definition will become

```
def f(request):
...
    user = call_handler(get_user, request)
...
```

`call_handler` will call the passed function with given arguments. Before calling it will start tracking for the SQL queries. And after the function has been called it will collect the sql queries and will store it with the function so that we can see later if during call any SQL query was made. Moreover, to attain the functionality of collecting queries we will have to patch the Django code which makes SQL queries (we will talk about it later). Its definition would be something like

```
def call_handler(func, *args, **kwargs):
    sql_counter.start_tracking()
    result = func(*args, **kwargs)
    sql_queries = sql_counter.collect_queries()
    store_queries(func, sql_queries)
    return result
```

And then we'll be able to see what calls inside function made SQL queries. This will be a really useful feature for optimizing code.

Implementation

How to manipulate AST?

The Python docs of AST doesn't give much information on its usage. Then we found Green Tree Snakes. Also we found an example to convert Python code to JavaScript in a Stack Overflow answer using AST transformation. We found them pretty helpful. There were following improvements in initial ideas while implementing: